

Late Breaking Results: Routability-Driven FPGA Macro Placement Considering Complex Cascade Shape and Region Constraints

Hao Gu¹, Jian Gu¹, Keyu Peng¹, Jun Yang^{1,2}, Ziran Zhu^{1,2*}

¹National ASIC System Engineering Center, Southeast University, Nanjing, China

²National Center of Technology Innovation for EDA, Nanjing, China

{230218727, 220216018, kypeng, dragon, zrzhu}@seu.edu.cn;

ABSTRACT

Field-programmable gate array (FPGA) macro placement holds a crucial role within the FPGA physical design flow since it substantially influences the subsequent stages of cell placement and routing. In this paper, we propose an effective and efficient routability-driven macro placement algorithm for modern FPGAs with cascade shape and region constraints. To reserve adequate space for cell placement and guarantee routability, we first develop a routability-driven mixed-size analytical global placement (GP) that evenly distributes both macros and cells while considering cascade shape and region constraints. Then, we propose an integer linear programming (ILP)-based cascade shape legalization (LG) followed by matching-based macro legalization to remove macro overlaps while satisfying the region constraints. Finally, a routability-driven detailed macro placement is proposed to refine the solution. Compared with the top contestants of the MLCAD 2023 contest, experimental results show that our algorithm achieves the best overall score and routability.

1 INTRODUCTION

Recently, the MLCAD 2023 held an FPGA macro placement contest and provided the benchmarks specifically designed for the Xilinx UltraScale+ architecture [1]. The MLCAD 2023 contest benchmarks not only impose the more practical constraints (i.e., cascade shape and region constraints) based on real-world applications, but also offer an expanded array of DSPs, block RAMs (BRAMs) and ultra RAMs (URAMs). As illustrated in Figure 1(a), each cascade shape constraint requires that specific macros be placed consecutively in a predetermined order. As illustrated in Figure 1(b), the region constraints mean that instances assigned to a region must be placed within that region. To address the FPGA macro placement, recent advanced works [3, 8] typically employ a mixed-size placement approach that places macros and cells. However, due to sharp increase in the number of macros and the necessity to account for additional cascade shape and region constraints, it is desirable to develop a practical placer.

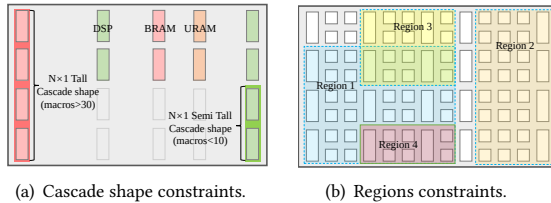


Figure 1: Example of cascade shape and region constraints.

* Corresponding author.

This work was partially supported by the National Key Research and Development Program of China under Grant 2022YFB4400500, and the National Natural Science Foundation of China under Grant 62104037.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06...\$15.00

https://doi.org/10.1145/3649329.3663497

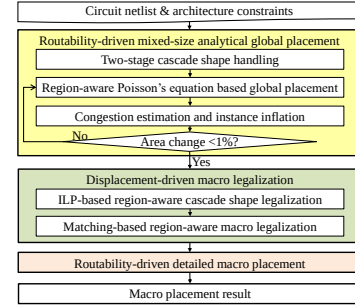


Figure 2: The overall flow of our proposed algorithm.

In this paper, we develop an effective routability-driven macro placement method for modern FPGAs with cascade shape and region constraints. Based on the contest's public benchmark suite [1], experimental results show that our algorithm achieves the best overall score and routability among all contest winners.

2 THE PROPOSED ALGORITHM

Figure 2 provides our algorithm flow, which mainly includes three parts: (1) routability-driven mixed-size analytical GP, (2) displacement-driven macro legalization, and (3) routability-driven detailed macro placement.

2.1 Routability-Driven Mixed-Size Analytical GP

To reserve adequate space for cell placement and guarantee routability, this subsection details our mixed-size analytical GP. If the total overflow area $O_t[3] < 0.25$ for macros and $O_t < 0.15$ for cells, we refer to this as the advanced stage of GP; otherwise, we regard it as the early stage.

2.1.1 Two-stage cascade shape handling. To ensure both the flexibility of instance for spreading and the adherence to cascade shape constraints, we propose a two-stage cascade shape handling technique based on different levels of instance spreading. In early stage, since instances are not being sufficiently spread, we introduce pseudo nets with higher net weights for each cascade shape to force the macros within the same cascade shape to be placed closer to each other. In advanced stage, since instances are broadly spread, macros within the same cascade shape constraint are merged as a single large cluster in predetermined order.

2.1.2 Region-aware Poisson's equation based GP. Similar to Pplace-MS[5], the system potential energy in this paper can be calculated by:

$$\mathcal{D}(x, y) = \sum_{b_i \in B} \sum_{t \in T} \iint_{R(b_i, t)} \psi_t(x, y) dx dy, \quad (1)$$

where $R(b_i, t)$ ($t \in T = \{LUT, FF, DSP, BRAM, URAM\}$) is the intersection region of instance b_i and the region of resource type t , and ψ_t is the electric potential of t . For handling region constraints, we have the following region tension function $F_{u_i}(x_i, y_i)$ of the region u_i for b_i :

$$F_{u_i}(x_i, y_i) = f_H(x_i, u_i) + f_V(y_i, u_i), \quad (2)$$

where $f_H(x_i, u_i)$ and $f_V(y_i, u_i)$ are horizontal and vertical region costs. Thus, region-aware FPGA global placement model can be formulated as:

$$\min_{x, y} g(x, y) = W(x, y) + \lambda_d \mathcal{D}(x, y) + \lambda_r F(x, y), \quad (3)$$

where λ_d and λ_r are regularization factors. And $F(x, y)$ is the total region tension of instances with region constraints.

Table 1: Experimental Results on MLCAD public benchmarks.

Metrics ¹	$TP\&R \downarrow$ (hr)		S_{IR}		$S_{DR} \downarrow$		$S_R \downarrow$		$S_{score} \downarrow$	
	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.
Ours	0.35	0.34	1.49	1.28	6.84	6.77	10.57	8.65	3.91	2.97
Team 1	0.36	0.35	1.57	1.28	7.34	7.22	12.86	9.16	5.40	3.22
	+3.2%	+2.3%	+5.8%	-0.1%	+7.4%	+6.5%	+21.6%	+5.9%	+38.1%	+8.3%
Team 2	0.38	0.37	1.56	1.30	7.09	6.99	11.88	9.09	5.30	3.36
	+10.4%	+7.7%	+5.3%	+1.7%	+3.8%	+3.3%	+12.4%	+5.1%	+35.6%	+13.2%
Team 3	0.37	0.36	1.83	1.36	7.23	7.11	15.27	9.67	7.76	3.47
	+6.9%	+4.4%	+23.1%	+6.5%	+5.7%	+5.0%	+44.5%	+11.8%	+98.5%	+16.8%

¹ Percentages indicate how much worse (+) or better (-). Team 1,2,3 are compared to ours.

A weighted average (WA) model $W(x, y)$ [8] is used to smooth the wirelength function. Then, based on $\nabla g(x, y)$, a preconditioned Nesterov's method in [2] is used to optimize the positions of movable instances. Although region force $\nabla F(x, y)$ effectively guides instances toward their corresponding regions during GP, there may still be several instances that fail to satisfy the region constraints. Therefore, during the advanced stage, we further impose a forced region shift on the instances with region constraints to ensure that all the region constraints are met.

2.1.3 Congestion estimation and instance inflation. In the advanced stage, similar to [3], we adopt rectangular uniform wire density (RUDY)[6] to estimate routing congestion and calculate estimated area A_i^{est} of b_i . The inflation area is defined as $\Delta A_i = A_i^{est} - A_i$, A_i is the current area of b_i . Then, the final area A_i^{update} of b_i after inflation can be calculated as $A_i^{update} = A_i + \tau_i \times \Delta A_i$, τ_i is a factor to prevent the total inflation area from exceeding the total area of each resource type.

2.2 Displacement-Driven Macro Legalization

2.2.1 ILP-based region-aware cascade shape legalization. To legalize the cascade shapes and handle multiple regions simultaneously, we formulate the region-aware cascade shape legalization as an ILP model in (4). C and S denote all clusters and sites respectively. Note that the macros subject to a cascade shape constraint are regarded as a cluster.

$$\min \sum_{c \in C} \sum_{g \in S_c} m_{cg} \cdot P_c \cdot d_{cg} \quad (4)$$

$$\text{s.t.} \quad \sum_{g \in S_c} m_{cg} = 1, \forall c \in C; \quad (4a)$$

$$\sum_{c \in C} n_{cs} \leq 1, \forall s \in S; \quad (4b)$$

$$\sum_{s \in g} n_{cs} = N_c \cdot m_{cg}, \forall c \in C, \forall g \in S_c; \quad (4c)$$

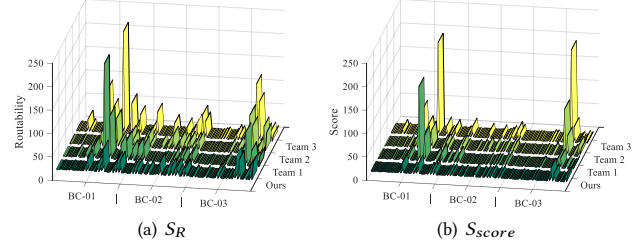
$$m_{cg}, n_{cs} \in \{0, 1\}, \forall c \in C, \forall g \in S_c, \forall s \in S; \quad (4d)$$

where S_c is the set of alternative groups for cluster c , each group g contains a set of sites for accommodating c , P_c is the preconditioner defined as [3], d_{cg} is the displacement of moving cluster c to group g , and N_c is the number of macros in cluster c . In the ILP model (4), the objective is to minimize the total weighted cluster displacement. Constraint (4a) denotes each cluster c can only choose an alternative group of sites. Constraint (4b) represents each site s can only be occupied at most once. Constraint (4c) denotes each cluster c occupies N_c sites since the cluster contains N_c macros. Given that modern FPGAs typically contain dozens of cascade shapes, the ILP model should be solved by Gurobi [4] for DSP, BRAM, and URAM separately.

2.2.2 Matching-based region-aware macro legalization. After the cascade shapes LG, all macros in cascade shapes are fixed. The LG of remaining macros can be solved as a min-cost bipartite matching problem.

2.3 Routability-Driven Detailed Macro Placement

The macros with cascade shape constraints are fixed in this stage. Based on RUDY[6], we first calculate the horizontal and vertical routing utilization H_i and V_i for each macro b_i . Then, we iteratively identify b_i with the highest $\max(H_i, V_i)$ to reallocation. In an attempt to reallocate b_i , the optimal region R_{opt} for b_i is determined by method described in [3].


Figure 3: Comparison of S_R and S_{score} in BC-01, BC-02 and BC-03.

If available sites exist in R_{opt} , we try to move macro b_i to an available site in R_{opt} . If R_{opt} lacks available sites, we try to swap b_i with the macro on an occupied site. Note that macro reallocation occurs only if it can reduce the maximum routing utilization while maintaining the legality of the macro placement. Once a macro is reallocated, it is set as fixed. When three attempts to reallocate different macros fail, this iterative macro reallocation process terminates.

3 EXPERIMENTAL RESULTS

We implemented the proposed macro placement algorithm in a combination of C++ and Python. Vivado 2021.1 [7] was used to verify the legality of macro placement and perform cell placement and routing. The number of macros on MLCAD 2023 contest public benchmarks ranges from 2400 to 2700, and the number of cells included ranges from 550k to 710k. We categorize the total of 140 benchmarks into three groups: “without cascade shape”, “with semi-tall cascade shapes”, and “with tall cascade shapes”, denoted as BC-01, BC-02, and BC-03, comprising 46, 46, and 48 benchmarks, respectively. According to the MLCAD 2023 contest [1], the final evaluation score S_{score} is defined as follows:

$$S_{score} = [1 + \max(0, T_{macro} - 10)] \times S_R \times TP\&R. \quad (5)$$

where T_{macro} denotes the runtime of macro placement in minutes, $TP\&R$ is the runtime of cell placement and routing performed by Vivado in hours, and S_R is the routability score defined as $S_{IR} \times S_{DR}$. S_{IR} represents initial routing score, and S_{DR} represents detailed routing score in [1].

In the experiment, we compare the proposed algorithm with the top three teams (i.e., UTDA, SEU, and MPKU-post) in the contest [1]. Table 1 gives the arithmetic mean (denoted by Avg.) and geometric mean (denoted by Geo.) values of the metrics $TP\&R$, S_{IR} , S_{DR} , S_R , and S_{score} . Considering T_{macro} of all teams remains below 10 minutes, preventing runtime penalty in S_{score} , we have not given it. Compared with other teams, our algorithm has achieved superior results in most metrics. In terms of S_{score} , our algorithm outperforms Team 1, Team 2, and Team 3 by 8.3%, 13.2%, and 16.8% in geometric mean. Besides, Figure 3 provides the detailed comparison of S_R and S_{score} for each benchmark in BC-01, BC-02 and BC-03. As can be seen from the figure, our algorithm achieves the best routability and final score in most benchmarks.

REFERENCES

- [1] I. Bustany, G. Gasparyan, A. Gupta, A. B. Kahng, M. Kalase, W. Li, and B. Pramanik. The 2023 mllcad fpga macro placement benchmark design suite and contest results. In *Proceedings of ACM/IEEE Workshop on Machine Learning for CAD*, pages 1–6, 2023.
- [2] J. Lu, P. Chen, C. C. Chang, L. Sha, and C. K. Cheng. ePlace: Electrostatics-based placement using fast fourier transform and nesterov's method. *ACM Transactions on Design Automation of Electronic Systems*, 20(2):1–34, 2015.
- [3] Y. Meng, W. Li, Y. Lin, and D. Z. Pan. elfplace: Electrostatics-based placement for large-scale heterogeneous fpgas. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(1):155–168, 2022.
- [4] J. P. Pedroso. Optimization with gurobi and python. *INESC Porto and Universidade do Porto, Porto, Portugal*, 1, 2011.
- [5] K. Peng and W. Zhu. Pplace-MS: Methodologically faster poisson's equation based mixed-size global placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.
- [6] P. Spindler and F. M. Johannes. Fast and accurate routing demand estimation for efficient routability-driven placement. In *Proceedings of Design, Automation and Test in Europe Conference*, pages 1–6, 2007.
- [7] Xilinx Corporation. Zynq ultrascale+ device technical reference manual. Online, 2022.
- [8] Z. Zhu, Y. Mei, Z. Li, J. Lin, J. Chen, J. Yang, and Y.-W. Chang. High-performance placement for large-scale heterogeneous fpgas with clock constraints. In *Proceedings of ACM/IEEE Design Automation Conference*, pages 643–648, 2022.